

3>SHAKE

システムは「動く」だけでは
足りない 実装編

非機能要件・分散システム・トレードオフをコードで見る

2026/04/17 TECH BAR @NUTIC co-created with 3-shake

@nwiizo 20min

今日お話しすること

基礎編で見た「守るものが違うと設計が変わる」を、今回は小さな Rust サンプルで確かめます。

1. **retry（やり直し）** が、なぜそのままだと危ないのかを見る
2. **レプリカ（複製側）を読むこと** が、なぜ少し古い値を連れてくるのかを見る
3. **その判断をどう残すか** までつなげる

題材にするコード:

[workspace_2026/samples/system-tradeoffs-lab](#)



基礎編から実装編へ

基礎編では、

- 守るものが違えば設計が変わる
- 分けると「成功したか不明」な場面が増える
- 最後は何を守るために何を引き受けるかを決める

という話をしました。

実装編では、その中でも特に

retry（やり直し）と idempotency（同じ依頼を2回やらない工夫）、

`primary`（元のデータ）と `replica`（複製データ）の読み分けをコードで小さく再現します。

狙いは、概念や文法を覚えることではなく、**なぜその実装が必要になるのかを手触りでつかむことです。**



サンプルの全体像

このサンプルは、現実のシステムをかなり単純化しています。

注文処理の系

- FakePaymentGateway
- CheckoutService
- OrderRequest

在庫参照の系

- InventoryStore
- primary
- replica

ポイントは、難しい仕組みを再現することではなく、何を守ると何が増えるのかだけをはっきり見せることです。



図で見るサンプルの全体像

注文処理の流れ

注文内容



注文を進める役



決済サービス役

タイムアウト（timeout）に見える失敗とやり直し（retry）を再現する

在庫参照の流れ

在庫を見る役

最新の在庫

少し前の在庫

速さと最新性の引っ張り合いを再現する

前半は「やり直して事故が起きる」話、後半は「速く読むと少し古いかもしれない」話です。



ファイル構成

```
workspace_2026/samples/system-tradeoffs-lab/  
├── Cargo.toml  
├── README.md  
└── src  
    ├── lib.rs  
    └── main.rs
```

- [lib.rs](#)
トレードオフの再現ロジック
- [main.rs](#)
実行時の見せ方



Rust を読むための最低限の見方

Rust 初学者の人は、まずこの4つだけ押さえれば十分です。

`struct` (構造体)

関連するデータをまとめる箱

`enum` (列挙型)

状態の候補を並べる型

`match` (分岐)

状態ごとに処理を分ける

`HashMap` (辞書)

「キー → 値」で覚える辞書

今回は Rust の細かい文法を覚えることが目的ではありません。何を記録して、どこで分岐して、どう安全にしようとしているかが読めれば十分です。



struct（構造体）は「ひとかたまりの情報」

struct は、ばらばらの情報を1つにまとめるための箱です。pub は「外からも見える」という印で、今回は気にしなくて大丈夫です。

注文内容の例

- 注文番号 (request_id)
- 金額 (amount)

課金記録の例

- どの注文か (request_id)
- いくら課金したか (amount)

大事なのは、別々の情報を「この話題に関するまとまり」として持てることです。



enum（列挙型）と match（分岐）

`enum` は「この中のどれか1つが起きる」と表すための型、`match` は「その状態ごとにどう振る舞うか」を決める書き方です。

今回の候補（enum）

- タイムアウト後に実は成功
- 一時的な失敗
- 成功

分岐（match）のイメージ

駅の案内表示と同じ。「遅延中なら待つ」「運休なら別ルート」のように**状態ごとに行動を変える**

「なんとなく失敗っぽい」ではなく、**失敗にも種類があり、状態ごとに対応を変える**と分けて考えるのが大事です。



1. やり直し（Retry）だけでは危ない

タイムアウトのあとに「成功したか分からない」が残ると、retry だけでは事故になる

最初の登場人物

最初に見るのはこの3つです。細かい文法は気にせず、**どんなデータが登場するか**だけ見てください。

```
pub struct OrderRequest {  
    pub request_id: String,  
    pub amount: u64,  
}  
  
pub struct ChargeRecord {  
    pub request_id: String,  
    pub amount: u64,  
}  
  
pub struct FakePaymentGateway {  
    steps: Vec<GatewayStep>,  
    charges: Vec<ChargeRecord>,  
}
```

`OrderRequest` は注文の内容、`ChargeRecord` は課金の記録、`FakePaymentGateway` はテスト用の決済サービスです。



コードに出てくる記号の読み方

このあとのコードで繰り返し出てくる記号をここで整理しておきます。

pub

「外から使ってよい」という印です。付いていないものはそのファイルの中だけで使います。読み飛ばして大丈夫です。

Vec<X>

X を何個でも順番に並べて持てる入れ物です。

`Vec<ChargeRecord>` なら「課金記録を0個以上、順番に持つ入れ物」です。

String

文字列です。注文番号（`"order-001"` など）を入れます。

u64

0 以上の整数です。金額や在庫数など、マイナスにならない数値を入れます。



まずは役割だけつかめばよい

ここからコードが出てきますが、最初は細かい文法を追わなくて大丈夫です。

注文内容

誰が何円ぶん買うかを書いたメモ

注文を進める役

そのメモを持って決済サービスに依頼する

決済サービス役

課金を実行して、結果を返す

見たいのは、誰が何をして、どこで認識のずれが起きるかです。



決済サービスの状態を `enum` と `match` で書く

このサンプルでは、決済サービスの状態を `enum` (列挙型) で表しています。

```
pub enum GatewayStep {  
    TimeoutAfterCommit,  
    TemporaryFailure,  
    Success,  
}
```

```
match step {  
    GatewayStep::TimeoutAfterCommit => { ... }  
    GatewayStep::TemporaryFailure => { ... }  
    GatewayStep::Success => { ... }  
}
```

読み方は単純です。

- `enum` (列挙型) は「起こりうる状態の一覧」
- `match` (分岐) は「その状態ごとにどう振る舞うか」

という対応です。



成功したのにタイムアウトに見える

```
match step {
  GatewayStep::TimeoutAfterCommit => {
    self.charges.push(ChargeRecord {
      request_id: request.request_id.clone(),
      amount: request.amount,
    });
    Err(GatewayError::Timeout)
  }
}
```

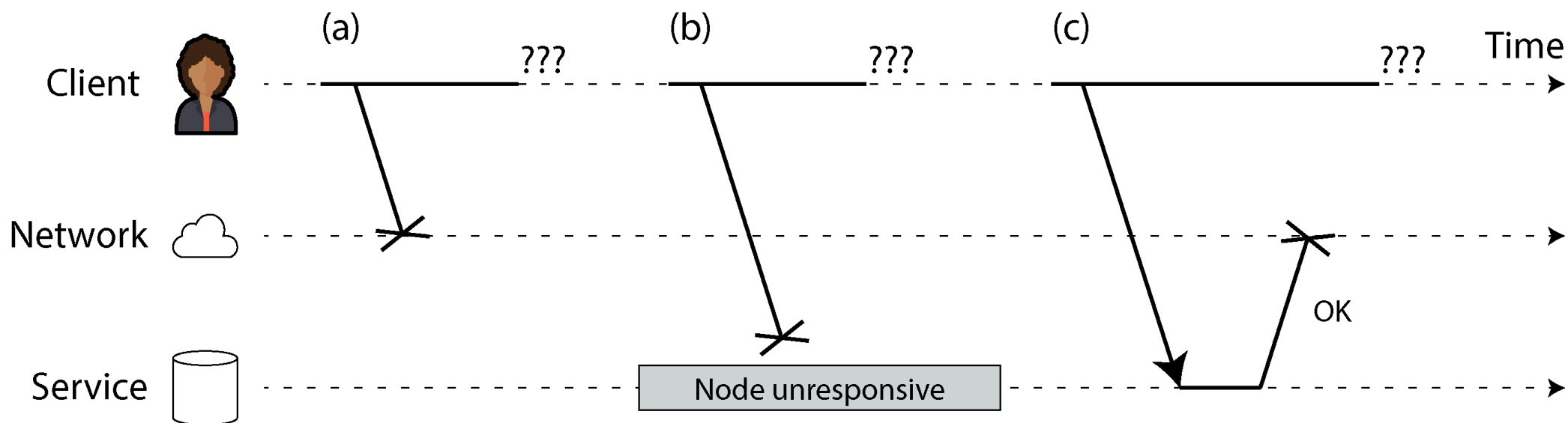
ここでやっているのは（`self` は「自分自身」、`.push( ... )` は「末尾に追加」、`.clone()` は「コピーを作る」）、

- `self.charges.push( ... )` で自分の課金記録リストに追加する（成功）
- でも呼び出し元には `Err(GatewayError::Timeout)` を返す（失敗に見える）

内部的には成功扱いの記録を残しているのに、返り値では失敗に見える状態です。



返事がないとき、何が起きたかは分からない



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 9-1 を引用

(a) リクエストが途中で消えた **(b)** 相手が止まっていた **(c)** 相手は処理したが返事が消えた — どれもこちらからは「返事がない」としか見えない。今回のコードで再現しているのは **(c)** のケースです。

だから retry するだけでは危ない

完全に失敗 (aやb)

課金もされない。やり直せばよい。

今回のケース (c)

課金は終わった。でも失敗に見える。

難しさは、失敗したことよりも、**成功したのか失敗したのか決めきれないこと**です。「分からない」ときにやり直すと、同じ処理を2回やってしまいます。



図で見る「失敗に見える成功」と二重課金

アプリ側

① 決済を依頼する

↓ 返事が来ない...

③ Timeout を受け取る
「失敗した」と判断

↓

④ もう一回依頼する (retry)

決済サービス側

② 課金記録を追加 
成功している

↓ でも返事が届かない

(待機中)

↓

⑤ 2回目も課金 → 二重課金

左右のずれが事故の原因。アプリは「失敗」と思っているが、サービスは「成功済み」

retry のコードを読む

CheckoutService (注文を進める役) はタイムアウトを見たら、素直にやり直します。

```
for attempt in 0..=self.max_retries {  
    match gateway.charge(&request) {  
        Ok(()) => return Ok(()),  
        Err(GatewayError::Timeout) => {  
            if attempt == self.max_retries {  
                return Err(CheckoutError::ExhaustedRetries);  
            }  
        }  
    }  
}
```

日本語で読むと: `for` で最大回数まで繰り返し → `gateway.charge` で課金を依頼 → 成功 (`Ok`) なら終了、タイムアウト (`Timeout`) なら次の回へ、回数を使い切ったら諦める。

このコードだけ見ると自然です。でも、前のスライドのように**実際は成功済み**だったら、同じ課金をもう一度してしまいます。retry は止まりにくさを上げますが、**課金や送信のように「やると結果が残る」**処理では別の安全策が必要です。



なぜ「もう一回」が自然に見えるのか

このときアプリの判断自体は不自然ではありません。

1. タイムアウトが返ってきた
2. 画面やプログラムには「失敗した」と見える
3. ならもう一回やってみよう、と考える

つまり問題は、**やり直し (retry)** という発想そのものではなく、**やり直しても安全かどうか**が分からないことです。分散では、この「分からなさ」を前提に設計する必要があります。



実行結果: 冪等性なしで retry した場合

```
scenario 1: retry without idempotency
result: Ok(())
charges: 2 / total_amount: 10000
```

読み方:

- `result: Ok(())` → 最終的には「成功」で終わった
- `charges: 2` → でも課金記録は **2件** できてしまった
- `total_amount: 10000` → 5000円の注文なのに **10000円** 引かれた

ユーザーから見ると「注文は1回」のつもりでも、決済サービスでは1回目で成功し、アプリは timeout だと思って再送し、2回目も成功して**二重課金**が発生しています。



ここで必要になるのが冪等性（べきとうせい / Idempotency）

冪等性（べきとうせい）は、retry をやめる工夫ではなく、成功したか失敗したか分からない場面でも事故を増やしにくくする工夫です。

考え方は単純です。「この `request_id` はもう処理した」と覚えておけば、同じ依頼が再送されても2回目は捨てられます。

ネットで買い物をして「購入」ボタンを押したのに画面が固まった。不安になってもう一度押した。でも課金は1回分だけ — **これが冪等性の効果**です。裏側では注文番号を見て「さっきと同じ依頼だ」と判断しています。



冪等性のコード: もう処理したか確認する

まず、同じ依頼が来たかどうかを確認します。`processed` は `HashMap` (辞書) で、「キーと値を対応づけて覚える箱」です。

```
if self.use_idempotency && self.processed.contains_key(&request.request_id) {  
    return Ok(());  
}
```

日本語で読むと:

1. `self.processed` → 自分が持っている「処理済みリスト」を見る
2. `.contains_key(&request.request_id)` → この注文番号はもう知っているか?
3. `return Ok(())` → 知っていたら、何もせず「成功」を返す



冪等性のコード: 処理したら記録する

初めて来た依頼なら課金して、`request_id` を記録しておきます。

```
self.processed
    .entry(request.request_id.clone())
    .or_insert(ChargeRecord {
        request_id: request.request_id.clone(),
        amount: request.amount,
    });
```

日本語で読むと:

1. `.entry(request.request_id.clone())` → この注文番号のところを開く
2. `.or_insert( ... )` → まだ記録がなければ、課金記録を書き込む
3. `.clone()` → 文字列のコピーを作る (Rust ではデータの所有権を意識する必要があるため)

これで、次に同じ `request_id` が来たときに「もう処理した」と判断できます。



なぜ `request_id`（リクエストID）が効くのか

`request_id`（リクエストID）は、注文1回ごとに付ける整理番号のようなものです。

整理番号がない

毎回「新しい依頼」に見えるので、再送で重複しやすい

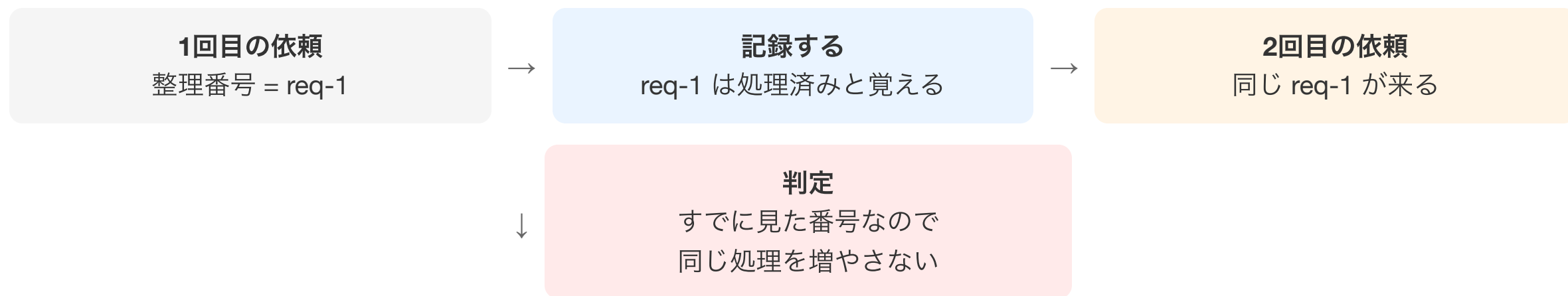
整理番号がある

同じ番号なら「前に見た依頼だ」と判断できる

実務では、注文ID、決済ID、メッセージIDなどがこの役割を持ちます。再送を安全にするには、「同じ依頼だ」と分かる印が必要です。

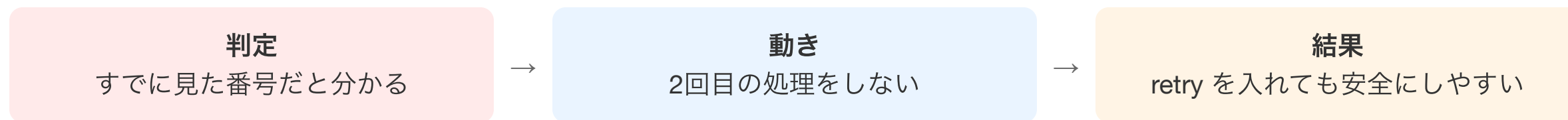


図で見る「整理番号で止める」



ポイントは、「もう処理した依頼だ」と判断できる記録を先に持つことです。

図で見る「2回目を防ぐ」



つまり request_id (リクエストID) は、再送そのものを止める印ではなく、同じ依頼が来てももう一度やらないための印です。

冪等性を入れた結果

```
scenario 2: retry with idempotency
result: Ok(())
charges: 1 / total_amount: 5000
```

読み方:

- `charges: 1` → 課金記録は **1件** だけ
- `total_amount: 5000` → 正しい金額のまま

retry はしたのに、2回目は `request_id` で「処理済み」と判断されて課金されていません。やり直し自体が悪いのではなく、やり直しても安全な仕組みをセットで入れることが大事です。



このコードで伝えたいこと

やり直し（retry）は止まりにくくするための道具

一時的な失敗から復帰しやすくなる。

冪等性（idempotency）は「成功したか分からない場面」で事故を増やしにくくする道具

やり直しによる二重実行を抑える。

retry だけでは足りない。分散では「止まりにくさ」と「二重実行しない」を組み合わせる

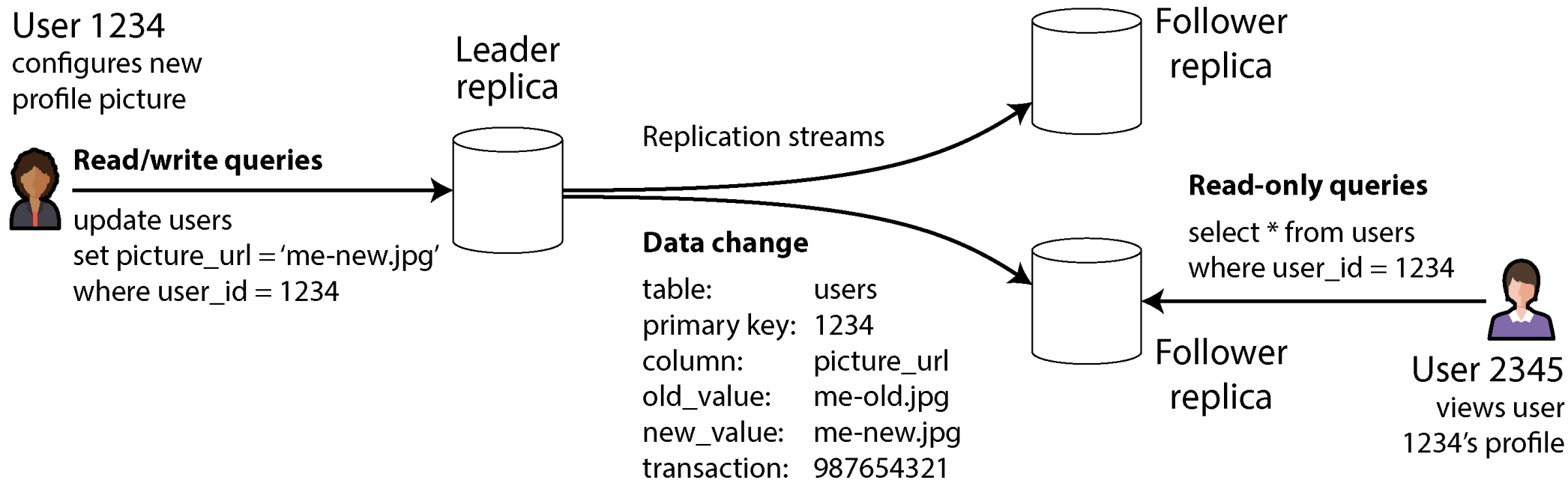
retry + 冪等性は「二重実行」という事故を小さくする話でした。次は、速さのために何を引き受けるかという別のトレードオフを見ます。



2. 速さと最新性は両立しにくい

速く読むための replica は、「少し古いかもしれない」を一緒に連れてくる

リーダーベースレプリケーションとは



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 6-1 を引用

書き込みは Leader（元データ）に行き、変更内容が Follower（複製）に流れます。読み取りは Follower からできるので速い。ただし、**流れが追いつくまでは古い値が返る**可能性があります。次のコードで、この仕組みを小さく再現します。



在庫のサンプル

```
pub struct InventoryStore {  
    primary: HashMap<String, Product>,  
    replica: HashMap<String, Product>,  
}
```

```
pub fn purchase_on_primary(&mut self, sku: &str, quantity: u32) {  
    if let Some(product) = self.primary.get_mut(sku) {  
        product.stock = product.stock.saturating_sub(quantity);  
    }  
}
```

このスライドで見たいのは、在庫データをどこに持っているかです。

ここで `saturating_sub` は、0より小さくならないように引き算する書き方です。

- `primary` (プライマリ) : まず更新される、元の在庫
- `replica` (レプリカ) : あとから追いつく、読むためのコピー



このサンプルで何を見たいか

このサンプルでは、

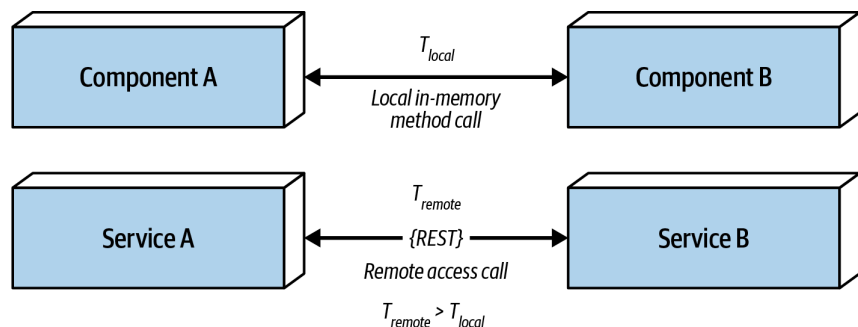
- 書き込みは `primary` (プライマリ)
- 読み取りは `primary` (プライマリ) または `replica` (レプリカ)

という、よくある構成だけを抜き出しています。見たいのは、**どこで速さを取り、どこで最新性を守るか**です。

ここでの `HashMap<String, Product>` は、
「商品IDを渡すと在庫データが返る」
というメモ帳のようなものだと思えば十分です。



まず「レプリカ」が何か



出典: Fundamentals of Software Architecture, 2nd Edition, Figure 9-8 を引用

レプリカ (replica) は、元のデータをコピーして持つ、読むための場所です。リモート呼び出しはローカルより遅いので、近くにコピーを置くと速く読めます。

primary (プライマリ)

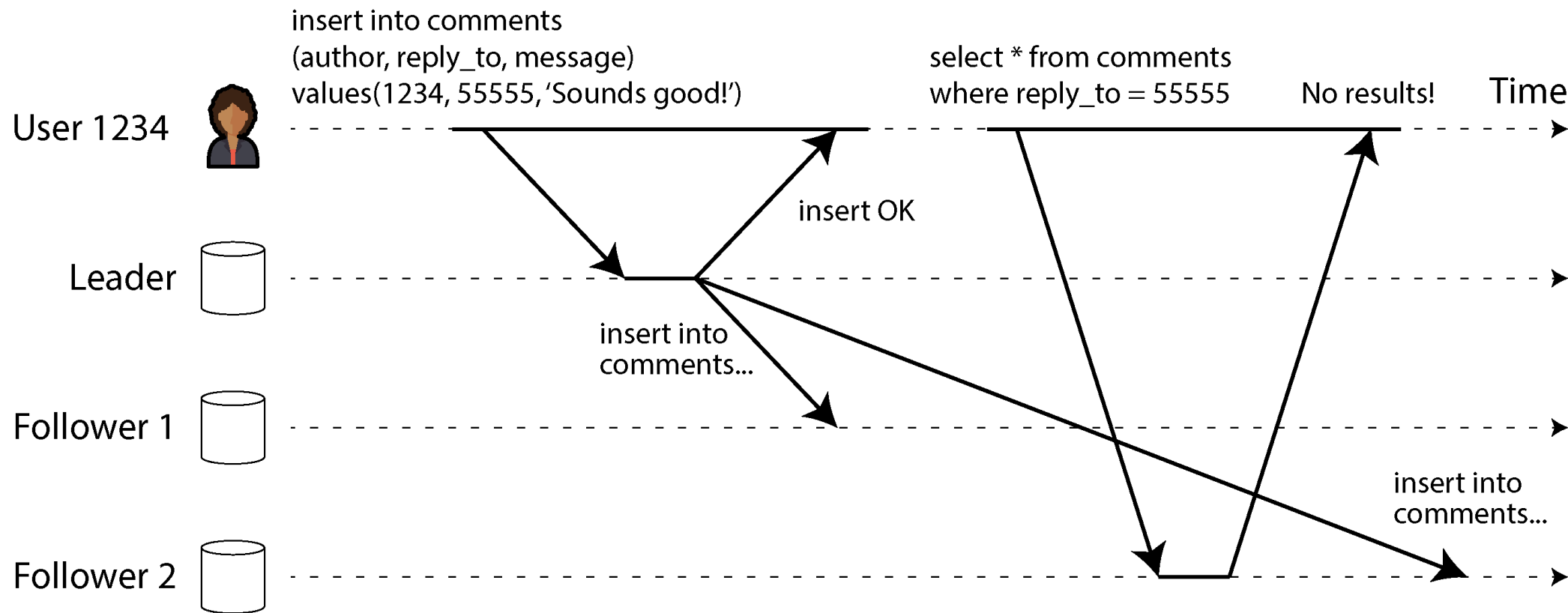
いちばん先に更新される元の在庫

replica (レプリカ)

あとから追いつく複製された在庫

レプリカは速く読むためのコピーですが、コピーなので少し遅れて追いつくと考えると分かりやすいです。

書いた直後に読むと、古い値が返ることがある



出典: Designing Data-Intensive Applications, 2nd Edition, Figure 6-3 を引用

User 1234 が Leader に書き込んだ直後、Follower から読むと「まだ反映されていない」状態が返ります。このサンプルの `primary` と `replica` は、まさにこの関係を小さく再現しています。

図で見る「場面で使い分ける」

一覧画面
`replica` でもよい



購入直前
`primary` を見たい

どちらが正しいかは、技術の好みではなく、**その場面で何を守りたいか**で決まります。



レプリカはすぐには最新にならない

```
pub fn replicate(&mut self) {  
    self.replica = self.primary.clone();  
}
```

この `replicate()` が呼ばれるまでは、`replica`（レプリカ）は古いままです。 `&mut self` は「自分自身を書き換えてよい」、`.clone()` は「まるごとコピーする」という意味です。

現実のシステムでも、

- 同期レプリケーション（複製が終わるまで待つ方式）なら待ちが増える
- 非同期レプリケーション（待たずに進める方式）なら古い値が見える

という両立しにくさがあります。速さを取りにいくほど、どこかで待つか、どこかで古さを受け入れるかの判断が必要になります。



ここで何を取って何をゆずるのか

レプリカ（replica）を使うと嬉しいこともあります。

得られるもの

読み取りを速くしやすい
混雑を分散しやすい

失うかもしれないもの

最新の値をすぐには見られない
場面によっては危険

だから、**いつもレプリカを読めばよい**のではなく、**どの場面なら少し古くてもよいか**を考える必要があります。速さを取るなら、その代わりに何を引き受けるかも一緒に決めます。

ここでも 1 回の流れを追う

1. 最初は `primary` も `replica` も在庫 `3`
2. `purchase_on_primary()` で `primary` だけ `2` になる
3. まだ `replica` には反映されないなので `3` のまま
4. `replicate()` を呼ぶと `replica` も `2` になる

つまり、`replica`（レプリカ）を読むというのは、**速さの代わりに「少し前の状態」を読む可能性を引き受けること**です。ここでも、守りたいものを1つ取ると、別の注意点が増えます。



実行結果: primary と replica の値のずれ

```
scenario 3: consistency vs latency
primary stock right after purchase: 2
replica stock before replication: 3
replica stock after replication: 2
```

読み方:

- `primary ... : 2` → 元データの在庫は購入後すぐ **2** に減った
- `replica ... before: 3` → 複製はまだ **3** のまま（古い!）
- `replica ... after: 2` → 複製処理の後にやっと **2** になった

購入直後なのに replica が `3` のままなのは、**反映がまだ終わっていない**からです。 `primary` と `replica` のどちらも間違いではなく、**どの場面で使うかが判断**になります。



どちらが正しいか

同じ在庫でも、正解は文脈で変わります。

商品一覧画面

多少古くてもよい。速さが大事。

購入確定直前

古い在庫は危険。最新性が大事。

ここで伝えたいのは、**正解が1つあるわけではなく、何を守りたいかで判断が変わる**ということです。



ここまでのまとめ: 基礎編との接続

この実装編で見たのは、基礎編のごく一部です。

基礎編の言葉	実装編で見たこと
タイムアウト (Timeout)	相手が成功しても、こちらからは失敗に見えることがある
やり直し (Retry)	止まりにくくできるが、状態が読めない場面では事故も増える
冪等性 (Idempotency)	retry を入れても二重実行しにくくする
一貫性 (Consistency)	<code>primary</code> は最新の値を返しやすい
遅延 (Latency)	<code>replica</code> は速いが、少し古いことがある

ここまで2つのトレードオフをコードで見てきました。最後に、こうした判断を「コードを全部自分で書かない時代」にどう活かすかを考えます。



3. コードを全部自分で書かない時代になぜ必要なのか？

コードが速く書けても、何を守るかと何を確かめるかは残る

実装が速くなっても、失敗は消えない

コーディングエージェントは、実装をかなり速く進められます。

速くなること

- 画面や API の雛形を作る
- テストコードをたたき台から書く
- リファクタリングを進める
- 定型的な実装をつなぐ

人が決めること

- どこまで retry をしてよいか
- timeout を本当に失敗とみなしてよいか
- 冪等性が必要な操作はどこか
- `primary` と `replica` をどの場面で使い分けるか

つまり難しさは、コードを書く量の問題ではなく、**どう壊れるかと何を守るかの問題**として残ります。実装速度が上がっても、失敗の形や優先順位までは自動で決まりません。



人に残る仕事は「決めること」と「確かめること」

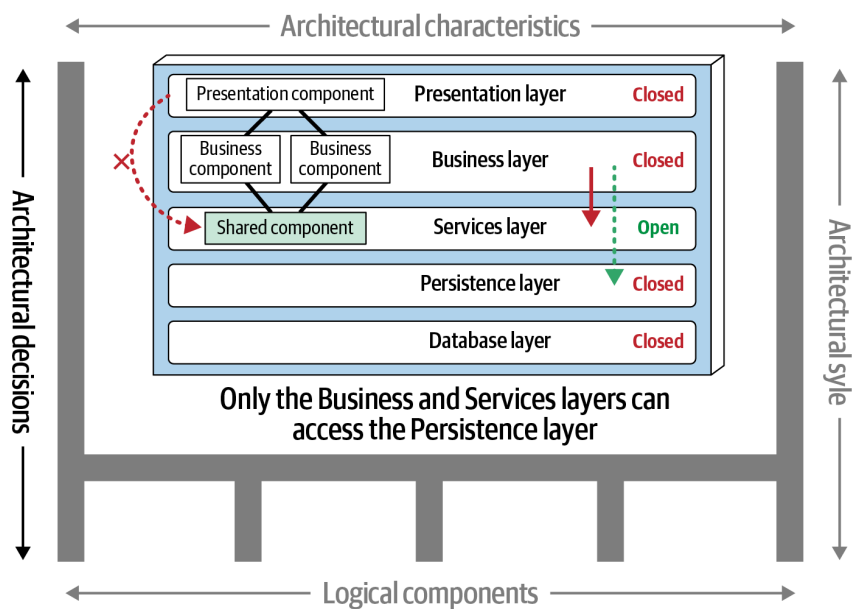
この実装編で見てきた話が重要なのは、エージェントが書いたものが良いかを見る基準になるからです。

1. `timeout` と `success` が食い違う場面を想像できる
2. やり直しが二重実行を生む危険を説明できる
3. `replica` の古さを許せる場面と許せない場面を分けられる
4. エージェントが書いた実装が、その考えに合っているか確かめられる

エージェント時代に価値が上がるのは、何を守るかを決めて、大きい事故を小さい不便に変えられているか確かめる力です



その判断を残すなら ADR がちょうどいい



出典: Fundamentals of Software Architecture, 2nd Edition, Figure 1-5 を引用

基礎編では「理由が言えない設計は、あとで弱くなる」という話をしました。コードを自分で全部書かない時代ほど、その理由を短く残しておく価値が上がります。

- なぜ やり直し (retry) を入れるのか
- なぜ `request_id` (リクエストID) で重複を防ぐのか
- なぜ `primary` と `replica` を分けるのか

こういう判断をあとから追えるようにするなら、**ADR (Architecture Decision Record)** が便利です。大げさな設計書ではなく、何を決めて、なぜそうしたかを短く残すためのメモです。

なぜ ADR が必要になるのか

設計の問題は、図だけ見ても理由が分からなくなることです。

図だけ残る場合

「なぜそうしたのか」があとで読めない

ADR がある場合

背景、決めたこと、その結果をあとから追える

つまり ADR は、**どんな形にしたか**よりも、**なぜその形にしたのか**を残すための道具です。理由が言えれば、変更するときにも判断をやり直せます。

ADR はどう書けばよいのか

最初から難しく考えなくて大丈夫です。まずは次の4つがあれば十分です。

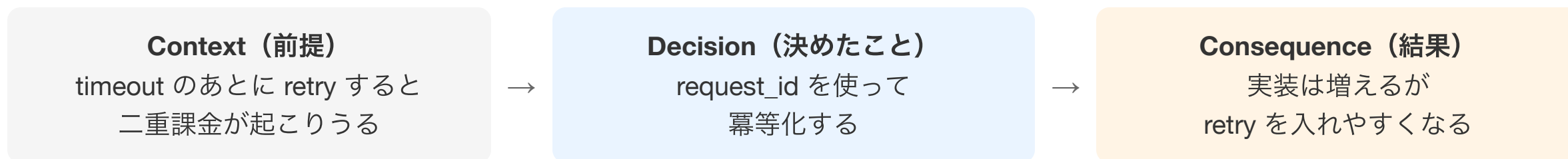
1. **Title (題名)** : 何を決める話なのか
2. **Context (前提)** : どんな困りごとがあるのか
3. **Decision (決めたこと)** : 何を選ぶのか
4. **Consequence (結果)** : 何がよくなり、何が増えるのか

Title: Retry 時の二重課金を防ぐため request_id を使う
Context: timeout のあとに retry すると同じ課金が重なることがある
Decision: request_id で同じ依頼かどうかを見分ける
Consequence: 実装は少し増えるが、二重課金の事故を減らしやすくなる

大事なのは完璧な文書にすることではなく、あとから読んだ人が「なるほど、この問題に対してこの判断をしたのか」と分かることです。



ADR も図で見るとシンプル



ADR は長い設計書ではなく、問題と判断と結果を短くつなぐメモだと考えると扱いやすいです。

次に触るなら

このサンプルの次の題材としては、次が自然です。

1. 一部だけ成功して、結果が分からなくなるケースをもっと詳しく再現する
2. 同期と非同期の違いを、順番に処理する仕組みで見る
3. ADR を書いて、何を守るための設計かを残す

設計を学ぶ近道は、壊れ方を小さく再現してみること



Rust 初学者として見るなら

今回のサンプルで読めるようになってほしいのは、全部ではなく次の3段階です。

1. `struct` や `HashMap` を見て、「どんなデータを持っているか」を読む
2. `enum` と `match` を見て、「どんな状態分岐があるか」を読む
3. 実行結果を見て、「この設計が何を守り、何を引き受けるか」を読む

文法を全部知ってから設計を学ぶのではなく、「何を守る実装か」を見ながら文法に慣れていけばよい



持ち帰ってほしいこと

コードは短くても、設計の問題は見える

大規模な本番システムでなくても、本質は小さなサンプルで観察できる。

便利な仕組みにも別の困りごとがある

retry（やり直し）も replica（レプリカ）も、それだけで安心とは限らない。

設計は「大きい事故」を「小さい不便」に変える仕事

どう壊れるかを先に考えて、何を守るために何を引き受けるかを決める。



ありがとうございました

ご質問・ご相談はお気軽にどうぞ

@nwiizo | <https://3-shake.com>